```
router = routers.SimpleRouter()
router.register(r'task',views.TaskView,base_name='task')
urlpatterns = [
    url(r'^',include(router.urls)),
    url(r'^register', views. RegistrationView.as_view(),
name='register'),
    url(r'^admin/', include(admin.site.urls)),
    url(r'^api-auth/', include('rest_framework.
urls',namespace='rest_framework')),
]
```

The explanation for various end points used in the code above is given below.

/task: We create a *SimpleRouter* object and register our view for automatic URL routing. This end point handles all CRUD operations related to the task object.

/register: This end point is defined for user registration. *RegistrationView* is Django class based views.

/api-auth: This is the built-in Django REST authentication end point.

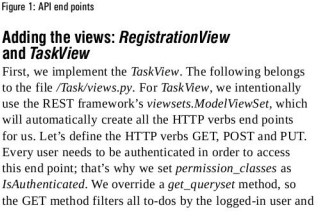

Figure 1: API end points

Adding the views: *RegistrationView* and *TaskView*

First, we implement the *TaskView*. The following belongs to the file */Task/views.py*. For *TaskView*, we intentionally use the REST framework's *viewsets.ModelViewSet*, which will automatically create all the HTTP verbs end points for us. Let's define the HTTP verbs GET, POST and PUT. Every user needs to be authenticated in order to access this end point; that's why we set *permission_classes* as *IsAuthenticated*. We override a *get_queryset* method, so the GET method filters all to-dos by the logged-in user and just responds with the serialised data. In the POST method, we validate the incoming data with the *TodoSerializer*. If the incoming data is valid, we create a to-do object and save it. The method replies with the incoming data and the primary key ID.

```
from rest_framework import permissions,viewsets
from .models import TaskModel
from rest_framework.generics import CreateAPIView
from .serializers import TaskSerializer,RegistrationSerializ
er
```

```
class TaskView(viewsets.ModelViewSet):
    """ Only Authenticate User perform CRUD Operations on
Respective Task
    """
    permission_classes = (permissions.IsAuthenticated,)
    model = TaskModel
    serializer_class = TaskSerializer

    def get_queryset(self):
        """ Return tasks belonging to the current user """
        queryset = self.model.objects.all()
        # filter to tasks owned by user making request
        queryset = queryset.filter(user=self.request.user)

        return queryset

    def perform_create(self, serializer):
        """ Associate current user as task owner """
        return serializer.save(user=self.request.user)
```

We use the *perform_create* method to associate the current user as the task owner, which is provided by the *mixin* classes, and provide easy overriding of the object *save*.

Now let's handcraft *RegistrationView*, which lets unregistered users register. The following belongs to the file */Task/views.py:*

```
class RegistrationView(CreateAPIView):
    """ CreateAPIView have only POST method
    """
    model = User
    serializer_class = RegistrationSerializer
    permission_classes = (permissions.AllowAny,)
```

We derive the *RegistrationView* from the *CreateAPIView* from the REST framework's generic views. *CreateAPIView* supports only the POST HTTP verb, and we want to post the username and password into our database in order to create a user. Everyone, logged in or not, should be able to use this view; therefore, we set the *permission_classes* to an *AllowAny*. First, let's validate the incoming data with the *RegistrationSerializer*. After validation, we can create the user.

Play with the browsable API

Start the development server of your *TaskAPI* project. By default, this starts an HTTP server on port 8000.

```
$ (env) python manage.py runserver
```

Because the REST framework provides a browsable API, feel free to interact with the API through the Web browser at *http://localhost:8000/*.

Continued on Page 66...